



说说sharding那些事儿

目录

- 背景&简介
- 分布式数据访问组件选型
- 全局ID
- 跨库查询
- 分布式数据访问组件基本设计
- 读写分离
- 数据平滑迁移

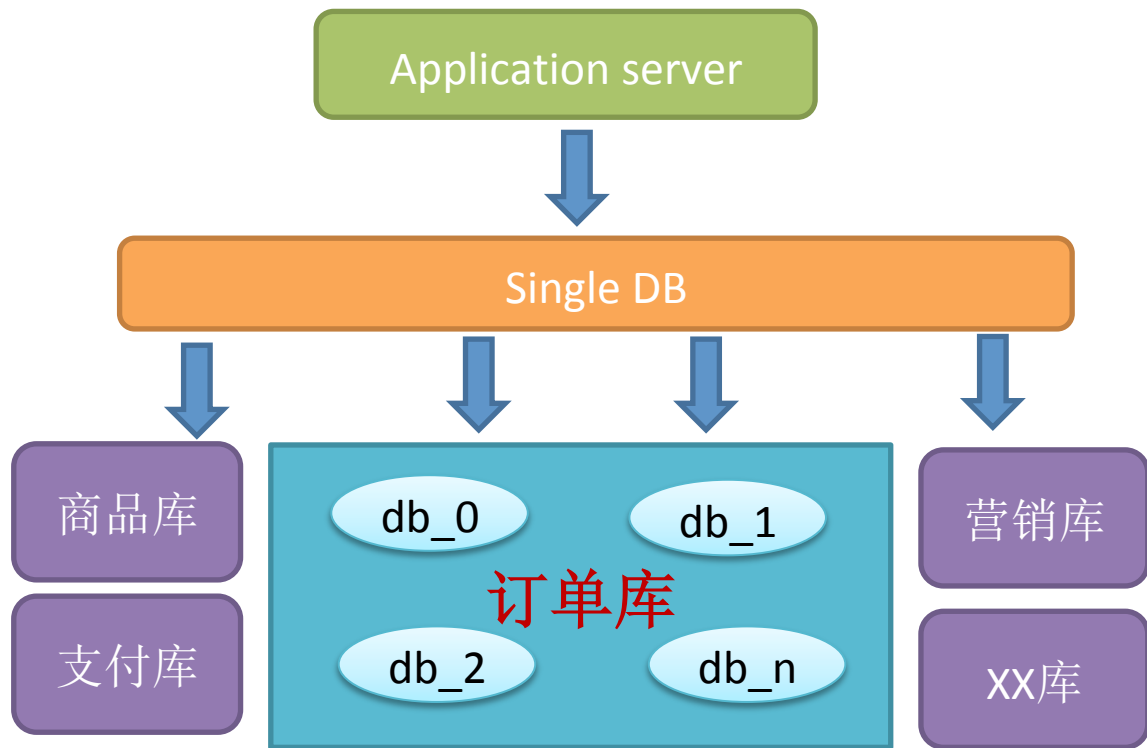
简介&背景

- 截止17-1，每天40W+订单，每月增长速度12%
- 归档后保留3个月数据，5KW订单，2Y菜品
- 当前RDS存储空间占用85%
- 解决方案
 - 订单库拆分 5月底6月初完成
 - 实施sharding 6月底7月初

简介&背景

- Sharding的基本思想就要把一个数据库切分成多个部分放到不同的数据库上，从而缓解单一数据库的性能问题
- 分为垂直和水平切分，这里主要讲后者

简介&背景



分布式数据访问组件

- PROXY方式
- 独立部署，大多数产品均对接入应用完全透明
- 多1次网络传输，需要额外的运维成本
- 典型如MYCAT
- 均以MYSQL为例



分布式数据访问组件

- SDK方式（嵌入JAR包）
- 随应用一起部署，依赖语言
- 对应用有一定地侵入和改造成本
- 直连数据库，性能较好
- 典型如sharding-jdbc（客如云目前的选型）



全局ID

- 水平切分场景中，需要对原单库中的自增ID进行改造
- 唯一性
- 连续性
 - 类似UUID算法简单高效，在整个分布式系统中连续性不好
 - 用一个集中式的单点服务来解决连续性问题

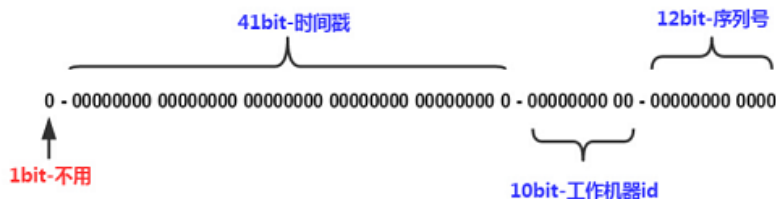
全局ID

- 全局ID生成器
- 高性能
 - 一次取一段ID缓存在本地
- 高可用
 - 无状态cluster
 - 底层存储有灾备措施

全局ID

- Twitter snowflake

snowflake-64bit

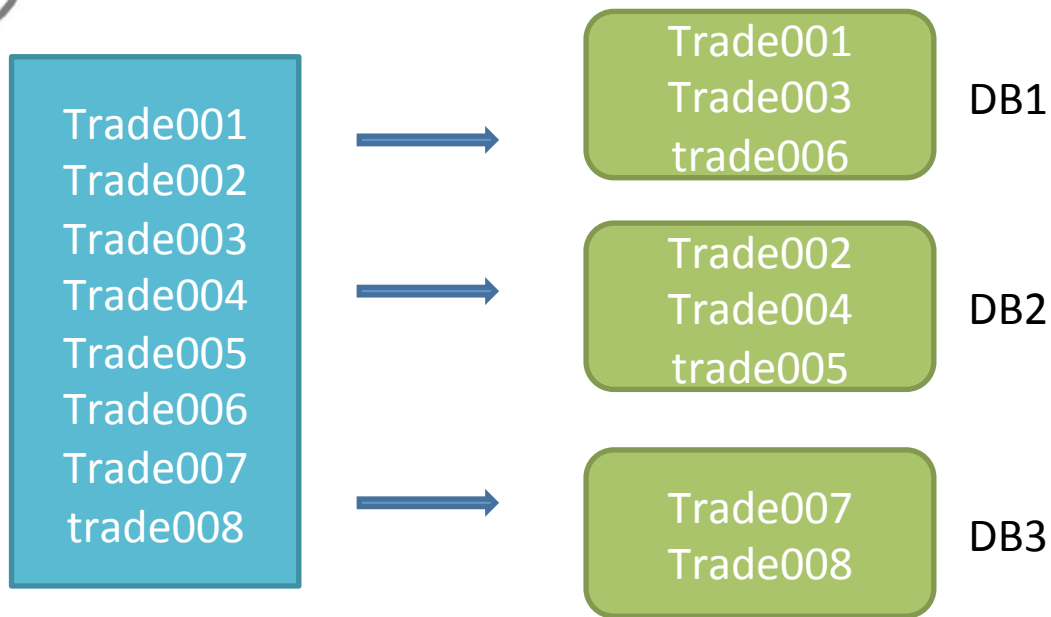


- Flickr :Mysql auto_increment + replace into + MyISAM
- Redis:lua script
- 芒果DB : objectId
- Others...

全局ID

- 客如云暂定方案
 - 64bit时间戳
 - 保证能用超过100年
 - 10bit进程ID
 - 需要在JVM参数中配置
 - 12bit自增序列号
 - 单个进程在1MS内限制生成次数，超过次数则等待1MS后重新生成

跨库查询



跨库查询

- 对多个库的查询结果进行合并
- 排序
 - 如果单个查询结果已排序，则对多个结果进行归并排序
 - 如果单个查询结果未排序，则对合并之后的结果集进行全排序
- 函数
 - Max、Min、Sum、Count
 - AVG : $\text{sum}(\text{每个查询结果的sum()}) / \text{sum}(\text{每个查询结果的count()})$

跨库查询

- 不排序分页
- 同等步长
 - 每页中来自不同DS的记录条数是一样的
 - DS1:1_1|1_2|1_3|1_4|1_5|1_6|1_7|1_8|1_9
 - DS2:2_1|2_2|2_3|2_4|2_5|2_6
 - 每页显示4条
 - Page1:1_1|1_2|2_1|2_2
 - Page2:1_3|1_4|2_3|2_4
 - Page3:1_5|1_6|2_5|2_6
 - Page4:1_7|1_8|1_9

跨库查询

- 不排序分页
- 同等比例
 - 每页中来自不同DS的记录条数的比例和不同DS的总数据条数比例一致
 - DS1:1_1|1_2|1_3|1_4|1_5|1_6|1_7|1_8|1_9
 - DS2:2_1|2_2|2_3|2_4|2_5|2_6
 - DS1:DS2=3:2
 - 每页显示5条
 - Page1:1_1|1_2|1_3|2_1|2_2
 - Page2:1_4|1_5|1_6|2_3|2_4
 - Page3:1_7|1_8|1_9|2_5|2_6

跨库查询

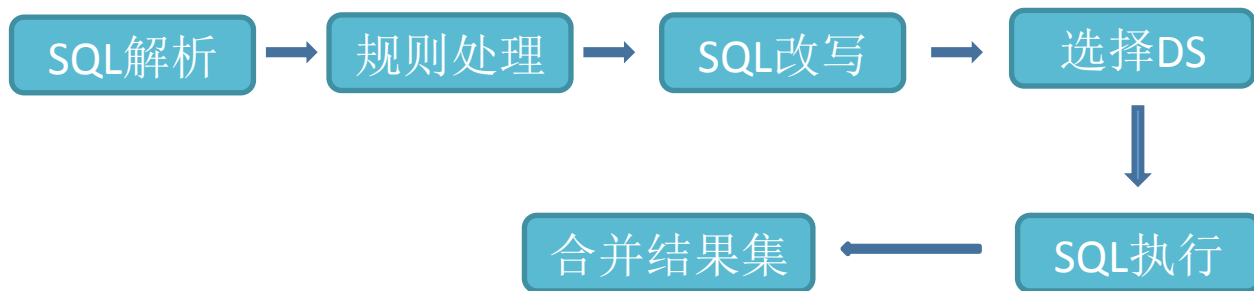
- 排序+分页
 - DS1:1|5|6|7|8|10|13|17|19|20
 - DS2:2|3|4|9|11|12|14|15|16|18
 - 每页4条数据，则第二页5/6/7/8均来自DS1
- 必须获取足够多的数据
 - 必须假设某一页的数据可能来自同一个DS
 - 对每个DS均获得当前页所有数据之后进行归并
 - 则第1页需要2个DS各自获取4条即共8条数据，然后进行归并排序
 - 假设1、2页的数据都来自于同个DS？
 - 则第2页需要2个DS各自获取自身前2页即 $4*2*2=16$ 条数据进行归并排序
 - 那么获得第N页所需要查询的数据条数为 $N*pageSize*dsCount$
 - Sharding场景尽量不要排序+分页（客如云的方案为不允许）



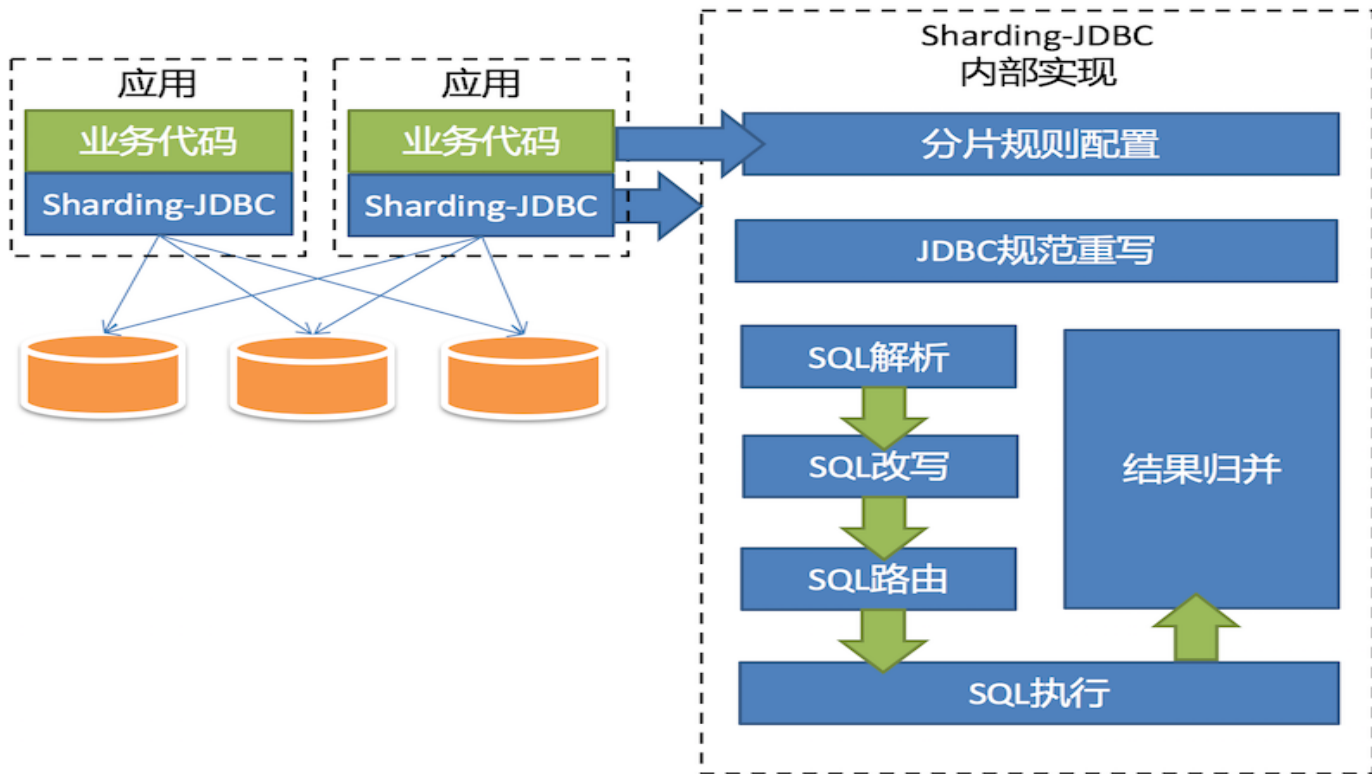
你TM在逗我

分布式数据访问组件基本设计

- 如果需要，如何制造轮子？
- 参考JDBC驱动实现



分布式数据访问组件基本设计



分布式数据访问组件基本设计

- SQL解析
 - 检查支持的SQL范围、方言
 - 获取关键信息（表名、分库条件字段）
 - 为后续可能的hint做准备（如读写分离）
 - antlr、javacc
- Sharding-jdbc
 - 不支持DDL、子查询
 - 不支持union
 - 不支持聚合distinct
 - 不支持dual(注意validate query如select 1)

分布式数据访问组件基本设计

- 规则处理
 - 根据一个或多个字段将数据分散到不同的库表中
- 固定HASH
 - 规则简单，但是扩容需要做数据迁移

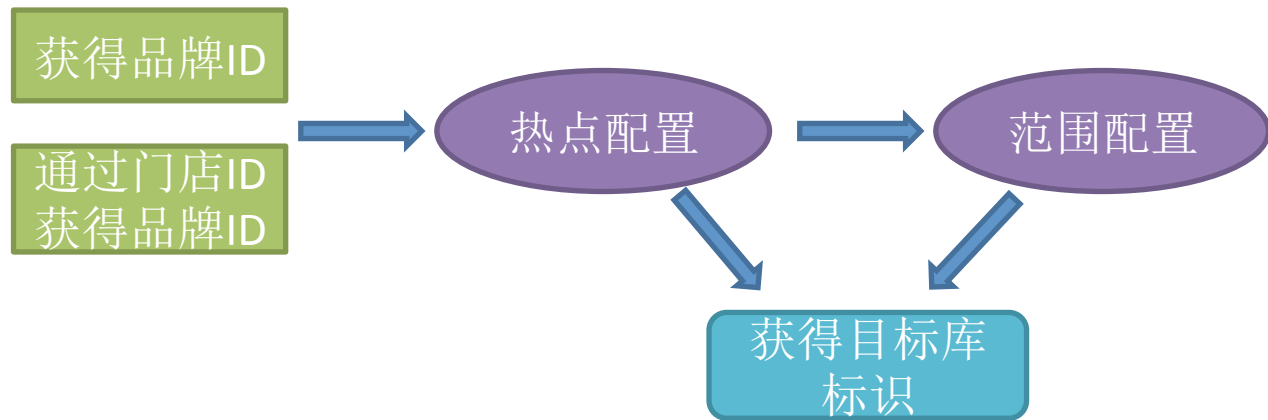
```
<rdb:strategy id="trade_database_Strategy"  
    sharding-columns="shop_identity"  
    algorithm-expression="dbms_${shop_identity.longValue() % 2}"/>
```

```
<rdb:strategy id="trade_Strategy"  
    sharding-columns="shop_identity"  
    algorithm-expression="trade_${shop_identity.longValue() % 4}"/>
```

- 一致性HASH

分布式数据访问组件基本设计

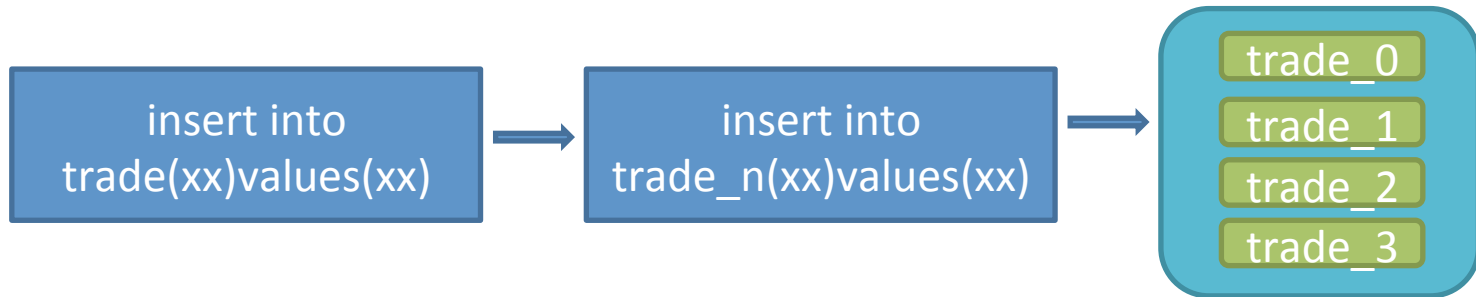
- 客如云的分片方案
 - 自定义规则（品牌ID范围）+指定热点用户
 - 配置本地化cache，程序+人工通知变更
 - 每个库中表名相同，不分表



分布式数据访问组件基本设计

- 改写SQL

- 如果每个库的表名均不同，则需要获得真实表名



```
<rdp:table-rule logic-table="trade" actual-tables="trade_{0..3}"
```

分布式数据访问组件基本设计

- 选择DS
 - 通过规则处理已经可以确定DS
 - 若有多个DS（多个从库、多主多从），这里要做的是LB
 - 轮询、权重
- SQL执行
 - 批量操作可选用线程池并行处理
- 合并结果集
 - 见“跨库查询”

读写分离

- 基于hint
 - 如sql中有写操作，则整个事务均路由到写库
- 强制路由
 - 切面中根据注解配置判断（客如云的方案）

平滑数据迁移

- 问题：迁移过程中仍然会有新的数据写入老库
- 停机迁移
 - 用户体验不好，对运维和研发压力大
 - 基于阿里云底层技术可以提前开始数据同步（类似双写），尽量缩短停机时间（[客如云商品库拆分实践](#)）
- 不停机迁移
 - 创建增量日志表，记录迁移过程中老库新增/修改/删除操作
 - 重新部署应用，开启向增量日志表写入老库的操作记录，同时开始迁移
 - 老库历史数据迁移结束，到新库中回溯执行增量日志表的数据操作
 - 到达业务低峰期回溯操作间断性执行完毕，新老库数据间断性完全一致
 - 修改配置逐个重启应用，关闭增量日志记录并切换到新库
 - 运维和研发压力小，最终步骤执行之前可以随时回滚或重头开始

平滑数据迁移

- 客如云方案
 - 配置自定义分片规则
 - 确保Sharding上线前的已有的商户订单数据全部存储在老订单库
 - 规避数据迁移
- Sharding实施完成之后如何实现动态扩容？

总结回顾

- 客如云实施sharding的背景和技术选型
- 实施sharding后需要面对的问题以及客如云的方案：全局ID、跨库查询、读写分离、平滑数据迁移
- 分布式数据访问组件基本设计思路及在客如云sharding方案中地体现



谢谢！

冯文

fengw@keruyun.com

2017-3

